

Repo0: Design-Driven Zero-to-All Code Generation

Silin Chen^{1,*}, Haoyi Teng^{1,*}, Xiaodong Gu^{1,†}, Yuling Shi¹, Jiale Huang¹, Yongpan Wang¹,
Hongyu Zhang², Haibing Guan¹

¹Shanghai Jiao Tong University

²Chongqing University

Abstract—Large language model agents have made substantial progress in code generation, yet most existing systems assume a predefined repository architecture. This assumption does not hold in zero-to-all code generation, where an agent must construct an entire software project directly from natural-language requirements while maintaining a modular repository architecture throughout development. We present Repo0, a continuous structural evolution framework for zero-to-all code generation. Repo0 maintains an explicit architectural state instantiated as a Dual-Directed-Acyclic-Graph (Dual-DAG), consisting of a requirement-level DAG, a component-level DAG, and their alignment relation. Starting from natural-language requirements, it iteratively evolves component boundaries through structural actions guided by modularity metrics until structural convergence, after which the converged architecture guides test-driven development code generation. We evaluate Repo0 on six real-world repositories from RepoCraft using GPT-5 mini and DeepSeek V3.2. Repo0 achieves the highest Functionality Coverage and Pass Rate across all settings. Compared with RPG, the strongest repository-planning baseline, Repo0 improves Functionality Coverage by up to 20.08 percentage points and Pass Rate by up to 29.74 percentage points. Ablation and structural-evolution analyses further demonstrate the importance of the Dual-DAG architectural state, modularity-guided structural evolution, and explicit structural convergence.

Index Terms—Software engineering agents, Repository Generation, large language models

I. INTRODUCTION

Code generation has seen remarkable progress with the advent of Large Language Models (LLMs), leading to highly capable agents that can synthesize complex code snippets and resolve repository-level issues [1]–[9]. However, current methods predominantly operate under a critical assumption: they assume the code has already been architecturally designed. In these structured repository completion settings, agents operate on top of predefined or partially specified repository architectures, where key design decisions, such as component boundaries, package organization, and dependency graphs, are largely given [10]. By focusing on the coding phase, these approaches overlook the crucial role of software architectural design.

In this paper, we focus on a more challenging and realistic problem: *zero-to-all code generation*. We seek to construct

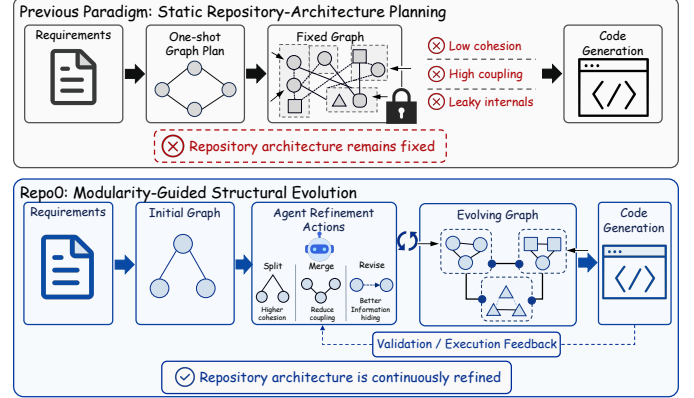


Fig. 1: Previous methods treat the graph as a fixed planning artifact, whereas Repo0 continuously evolves repository architecture during generation.

software projects without pre-existing repository architecture [11], [12]. This problem is difficult because the agent must jointly infer both the software’s functionality and its architecture. A well-designed software project must adhere to core software engineering principles such as high cohesion and low coupling [13]. When a coding agent attempts zero-to-all code generation without explicit architectural guidance, it could violate these modularity goals, resulting in inconsistent component boundaries, fragile dependency structures, and poor cross-file coordination. This suggests that the core bottleneck in zero-to-all code generation is not merely synthesizing code, but establishing and maintaining robust repository modularity throughout the development process.

Recent approaches have begun to explore this problem [10]–[12], [14], [15]. Approaches like NL2Repo-Bench [14] bypass the design stage by providing a golden repository architecture as part of the input. Multi-agent systems attempt to coordinate planning and implementation through specialized roles [16]–[18]. Recent graph-based approaches introduce explicit repository planning [11]. Despite these advances, these approaches suffer from a critical bottleneck: they treat software design as a *static* artifact. They assume that a perfect architectural blueprint can be generated during a single, initial planning stage and then rigidly executed. In real-world software development, however, project architecture is rarely ready during initial planning. As development progresses, the generated code often reveals whether planned components are truly cohesive or if dependency edges are overly entangled. Emerging complexities, duplicated functionalities, and validation

*Silin Chen and Haoyi Teng contributed equally.

†Xiaodong Gu is the corresponding author.

Emails: csisolow@gmail.com, chunemeng@outlook.com, xiaodong.gu@sjtu.edu.cn, yuling.shi@sjtu.edu.cn, unicornshjl@gmail.com, frankile@sjtu.edu.cn, hbguan@sjtu.edu.cn, and hongyujohn@gmail.com.

¹Our code and data are available at <https://github.com/csisolow/Repo0>

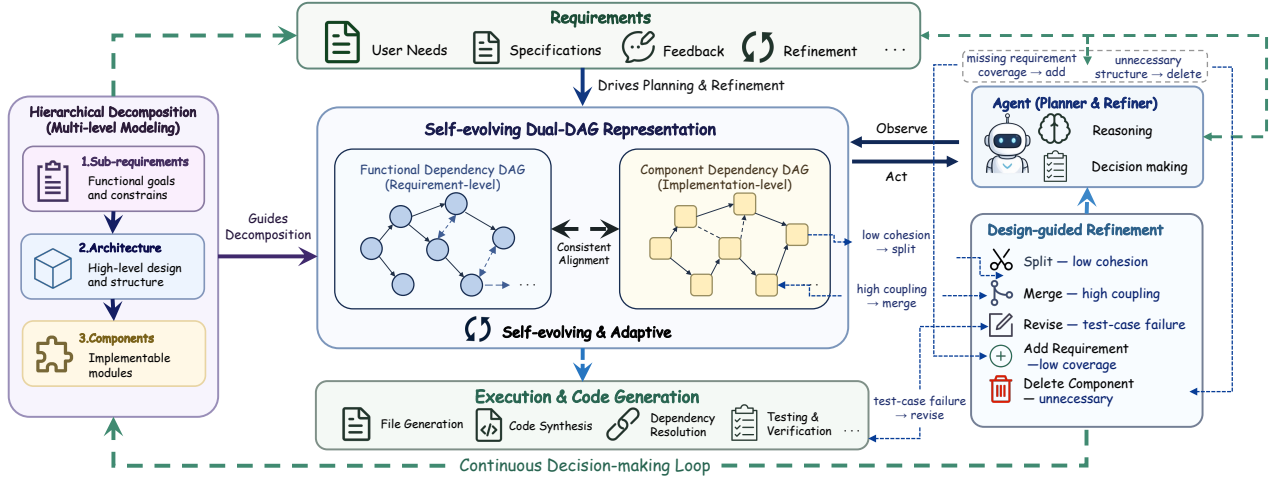


Fig. 2: Overview of the continuous decision-driven structural evolution framework.

feedback frequently necessitate splitting low-cohesion components or merging redundant ones. Consequently, zero-to-all code generation cannot be treated as a one-shot planning problem; it must be a continuous structural evolution process (as illustrated in Figure 1).

To address this limitation, we propose Repo0, a design-driven structural evolution framework for zero-to-all code generation. Rather than committing to a rigid initial blueprint, Repo0 explicitly models repository generation as a continuous structural evolution problem. Specifically, we maintain an architectural state represented as a Dual-Directed-Acyclic-Graph (Dual-DAG). The state separates requirement-level functional relationships from implementation-level components and dependencies, allowing functional coordination and implementation dependencies to evolve without being conflated in a single static planning graph. This Dual-DAG preserves traceability from the initial user prompt down to the code, while keeping the project architecture open to subsequent structural updates. Starting from a blank state with only natural language requirements, Repo0 executes a full-lifecycle software development process. It first extracts, normalizes, and decomposes the requirements to construct a requirement-level DAG. Then, it incrementally evolves the component-level DAG before code generation and uses the converged architecture to guide repository generation. Through explicit structural actions (i.e., add, split, merge, revise, and save), Repo0 dynamically adjusts component boundaries based on modularity metrics. After structural convergence, validation feedback drives localized repair during code generation. This evolution process is guided by modularity metrics and continues until structural convergence is reached. By treating design and implementation as deeply intertwined, Repo0 represents a paradigm shift in how autonomous agents construct complex software systems from scratch.

We evaluate Repo0 on RepoCraft [11], [19], a repository-generation benchmark consisting of six real-world Python repositories, under both GPT-5 mini and DeepSeek V3.2.

Results show that Repo0 consistently improves repository-generation quality over baselines. Across all six repositories and both backbone models, Repo0 achieves the highest Functionality Coverage and Pass Rate in all settings. Compared with RPG [11], the strongest repository-planning baseline, Repo0 improves Functionality Coverage by 4.55–20.08 percentage points and Pass Rate by 7.61–29.74 percentage points. Ablation results show that each major design component contributes to end-to-end performance, while the structural-evolution analysis further indicates that metrics-guided convergence is crucial: cohesion- and coupling-guided evolution improves repository quality, whereas unconstrained LLM-decided structural actions tend to over-decompose the repository architecture and degrade downstream correctness.

Overall, this paper makes three contributions:

- 1) We identify a fundamental limitation of existing repository generation methods: they treat repository architecture as a static artifact determined before implementation. We argue that repository modularity is only partially observable from requirements and emerges throughout coding. We formulate repository generation inherently as a continuous structural evolution problem rather than a one-shot planning problem.
- 2) We reformulate zero-to-all repository generation as a modularity-driven structural evolution process. Instead of committing to a fixed repository plan, repository architecture is continuously evolved through explicit structural actions guided by modularity metrics and requirement-coverage checks until structural convergence is reached. After convergence, validation feedback drives localized repair during code generation.
- 3) We instantiate this formulation in Repo0, a design-driven repository-generation framework built upon a Dual-DAG representation that separates requirement-level functionalities from implementation-level components and dependencies while maintaining end-to-end traceability across requirements, architecture, and code.

II. METHODOLOGY

A. Problem Formulation

We frame zero-to-all repository generation as a problem of continuous structural evolution. Unlike prior methods that first design a repository architecture and then treat it as a fixed blueprint for code generation, Repo0 treats repository design as an adaptive process whose structure is continuously revised as new architectural evidence emerges during requirement decomposition, component construction, code generation, and validation. The central objective is not only to capture complex long-context relationships across a repository for correct code generation, but also to establish and improve repository modularity throughout the development lifecycle.

Figure 2 illustrates the overall framework. To support continuous structural evolution, Repo0 maintains a persistent architectural state $S_t = (G_t^R, G_t^C, \mathcal{A}_t)$ for each time step t .

$G_t^R = (V_t^R, E_t^R)$ is a requirement-level DAG where V_t^R represents a requirement unit, including both high-level requirements and their sub-requirements. Edges in this DAG encode requirement-level functional coordination relations instead of implementation dependencies: an edge $(u, v) \in E_t^R$ indicates that two requirements are logically used together and should therefore be considered jointly when aligning their behavior, inputs, and outputs.

$G_t^C = (V_t^C, E_t^C)$ is the component-level DAG, where each $v \in V_t^C$ represents a component, such as a module, object, service layer, parser, or adapter, that can later be materialized as concrete source code. A component is the basic design unit that represents a bounded implementation responsibility (e.g., a reusable “Unified Modeling API”) that will later be materialized into concrete repository artifacts. E_t^C encodes implementation dependencies such as inheritance, reuse, or containment.

$\mathcal{A}_t \subseteq V_t^R \times V_t^C$ denotes the relations between requirements and components. A pair $(q, c) \in \mathcal{A}_t$ indicates that component c realizes all or part of requirement q . The relation is many-to-many: a requirement may require several components, and a reusable component may support multiple requirements. The alignment relation preserves traceability from requirements to components throughout structural evolution.

S_t is dynamic over time t : while the requirement-level DAG G_t^R generally stabilizes early, the component-level DAG G_t^C and the alignment relation $\mathcal{A}_t$ are continuously updated until modularity is optimized.

B. Phase I: Requirement Decomposition and Initial Architecture

The evolution process begins by translating a natural-language requirement document D into the initial architectural state S_0 . The requirement documents are usually uneven in granularity: some phrases describe broad project goals, while others describe constraints, behaviors, or usage scenarios. Directly mapping such descriptions to design modules often leaves important requirements under-specified and encourages brittle repository architectures.

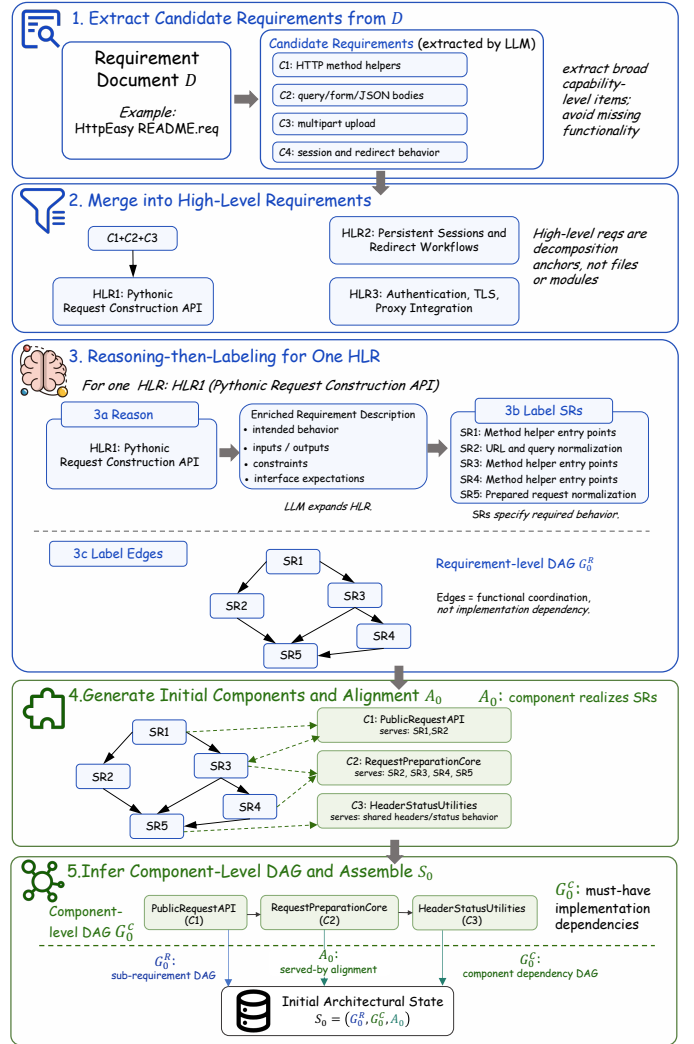


Fig. 3: Illustrative construction of the initial architectural state.

To resolve this mismatch, Repo0 first extracts candidate requirement items from D . Specifically, an LLM is prompted to identify capability-level requirements under the following regularizations: each item should describe a distinct repository-level functionality or system constraint, include the relevant operations and sub-features in its description, and avoid splitting low-level operations into separate top-level items. Repo0 then applies a requirement-merge step that consolidates redundant or subsumed items while keeping merely related requirements separate. The remaining merged items are treated as high-level requirements. They serve as top-level decomposition anchors rather than direct file or module specifications.

Next, each high-level requirement is decomposed into sub-requirements. A *sub-requirement* refers to a requirement-side functional unit that clarifies *what* functionality must be provided without determining packages, files, classes, or component dependencies. Inspired by Atom of Thoughts [20], Repo0 decomposes requirements through a three-stage reasoning-then-labeling process: First, an LLM is prompted to produce

as comprehensive a requirement description as possible, elaborating the intended behavior, inputs and outputs, functional constraints, interface expectations, error cases, and ambiguous scope. Based on this enriched requirement description, Repo0 then prompts the LLM to identify sub-requirements and finally label their logical dependency relations. A directed edge from sub-requirement u to sub-requirement v is added when v logically follows u , such as when v relies on the behavior, output, data contract, or interface assumption established by u . The resulting nodes and labeled edges form the local requirement-level DAG for that high-level requirement. Together, the high-level requirements and their decomposed sub-requirements form the initial requirement-level DAG, G_0^R .

Once G_0^R is constructed, the system derives the initial components to populate G_0^C and establishes the initial alignment relation $\mathcal{A}_0$. Concretely, for each high-level requirement, Repo0 collects its sub-requirements from G_0^R and prompts the LLM to translate them into a set of bounded components. Each proposed component contains a name, a responsibility description, and an explicit list of served sub-requirements. The proposed components form the initial component nodes V_0^C , and each served-sub-requirement entry creates an alignment pair $(q, c) \in \mathcal{A}_0$. Finally, Repo0 prompts the LLM to infer the implementation dependencies among the proposed components. Requirement-level coordination edges in G_0^R are provided only as soft evidence rather than being directly copied into G_0^C . The inferred dependencies initialize E_0^C . This initialization separates requirement-side artifacts from implementation artifacts.

Figure 3 illustrates how Phase I builds the initial architecture for a zero-to-all *HttpEasy* repository. Starting from D , Repo0 first extracts candidate requirement items and merges overlapping or subsumed items into high-level requirements. In this example, request construction, session persistence, response handling, and authentication are consolidated into two high-level requirements: “HTTP request and response core” and “session and authentication management”. For a selected high-level requirement, the LLM then produces an enriched semantic specification that elaborates parameters, headers, body encoding, timeout behavior, error handling, and response objects. Based on this specification, it identifies sub-requirements including constructing a request object, sending the request through a transport adapter, and parsing the returned response, and organizes them into a functional coordination chain from request construction to transport execution and response parsing, thereby instantiating G_0^R . These sub-requirements are then grounded into implementation components such as *RequestBuilder*, *TransportAdapter*, and *ResponseParser*. For instance, *RequestBuilder* serves the sub-requirement of constructing a request object, which establishes a corresponding entry in the alignment relation $\mathcal{A}_0$. Finally, Repo0 infers must-have implementation dependencies among components—for example, *SessionClient* depends on *RequestBuilder* and *TransportAdapter*—to construct G_0^C and yield the initial architectural state $S_0 = (G_0^R, G_0^C, \mathcal{A}_0)$.

C. Phase II: Modularity-Guided Structural Evolution Loop

The initial architectural state provides an initial assignment of requirement nodes to components, but may still be overly broad, fragmented, or redundant. Repo0 therefore evolves G_t^C and $\mathcal{A}_t$ through an evolution loop. During this phase, G_t^R is fixed to preserve the functional scope, while G_t^C and $\mathcal{A}_t$ are updated to improve repository modularity. The evolution is realized through four component-level structural actions:

- **split**: For a diffuse component c , Repo0 extracts the induced subgraph of G_t^R over its served sub-requirements $S(c)$ and uses graph partitioning [21], viewed as a minimum-cut objective [22], to identify candidate sub-requirement groups. The partition evidence is provided to the LLM, which rewrites c into a set of narrower components, redistributes the corresponding alignment pairs in $\mathcal{A}_t$, and reconnects incident dependencies in G_t^C .
- **merge**: Consolidates two components into a single component by combining their responsibility descriptions, merging their alignment pairs in $\mathcal{A}_t$, and redirecting incident dependencies in G_t^C to the merged component.
- **revise**: A boundary-preserving action that rewrites a component’s responsibility description, interface assumptions, implementation notes, or alignment entries.
- **save**: Marks a component as structurally stable and carries it forward unchanged in the current round unless neighboring structural updates make it eligible again.

These structural evolution actions are triggered by two modularity metrics and two auxiliary criteria:

a) *Responsibility*: Responsibility identifies the sub-requirements assigned to a component. For a component c , we define

$$RS(c) = \{q \in V_t^R \mid (q, c) \in \mathcal{A}_t\}. \quad (1)$$

The size $|RS(c)|$ measures how many requirement-side units that c is responsible for.

b) *Cohesion*: Cohesion identifies components that group diffuse, unrelated responsibilities [23]. Let $E_{in}(c)$ denote the number of edges in G_t^R whose endpoints both lie strictly within $RS(c)$. We define cohesion as the density of realized requirement-level functional coordination relations among the requirements grouped into the same component:

$$\text{cohesion}(c) = \begin{cases} 1, & |RS(c)| \leq 1 \\ \frac{E_{in}(c)}{|RS(c)|(|RS(c)| - 1)/2}, & |RS(c)| > 1 \end{cases} \quad (2)$$

Low cohesion indicates a structurally diffuse component.

A **split** action is triggered when a component’s cohesion falls below a threshold γ_{split} and the size of $RS(c)$ exceeds a threshold $\tau_{\text{split}}^{(t)}$. Here, $\tau_{\text{split}}^{(t)}$ is used to prevent the loop from over-splitting.

c) *Coupling*: Coupling identifies pairs of components that realize highly overlapping sets of sub-requirements [23]. Let RS_A and RS_B be the sub-requirement sets for which

components A and B are responsible, respectively. Coupling is defined as the Jaccard Similarity [24] between these sets:

$$\text{coupling}(A, B) = \frac{|RS_A \cap RS_B|}{|RS_A \cup RS_B|} \quad (3)$$

d) Connectivity: Connectivity measures the edges in G_t^R bridging RS_A and RS_B . A component pair is considered a merge candidate when their coupling exceeds a threshold θ_{merge} and connectivity is greater than 1. These metrics alone may neglect duplicated component responsibilities from valid architectural coupling, such as layered, adapter, or upstream-downstream relationships. Therefore, we ask an LLM to validate the candidates and determine the final merge action.

The structural evolution loop proceeds iteratively. In each round, Repo0 recomputes the modularity metrics and auxiliary criteria, then applies eligible structural actions to G_t^C .

The iteration terminates when a complete evolution round yields no eligible `split` or `merge` actions. At this point, structural convergence is reached with respect to component boundaries. The system then performs a final semantic alignment pass, where the LLM examines each component together with its aligned sub-requirements and neighboring components in the component-level DAG. If inconsistencies are identified between component responsibilities, requirement coverage, or interface assumptions, `revise` actions are applied.

D. Phase III: Code Generation

Once structural convergence is reached, the repository architecture is fixed, and Repo0 enters a code generation phase. The system first converts the converged G_t^C into a concrete generation plan: it derives package assignments, planned file paths, exported symbols, and dependency-aware generation order from G_t^C . For each component, Repo0 constructs a generation context from three sources: the component’s responsibility description, its aligned requirement nodes recorded in $\mathcal{A}_t$, and upstream components that must be available before the current component is implemented. This context specifies the component’s target behavior, placement, public symbols, and reusable dependencies.

Code is generated incrementally under a Test-driven development (TDD) workflow [25], [26]. For each component, Repo0 first generates an importable skeleton that fixes the expected public API, then synthesizes tests from the aligned requirement nodes and interface assumptions, and finally fills in the implementation to satisfy those tests. After each generation step, the system runs validation checks, including import checks, interface checks, and pytest-based execution. Failures such as missing symbols, incompatible call signatures, or unmet behavioral expectations trigger localized repair patches to the affected implementation, tests, or package initialization files. When validation feedback reveals that the component description or interface assumptions are inaccurate, Repo0 can apply `revise` before the next TDD workflow.

III. EXPERIMENTAL SETUP

A. Research Questions

We study three research questions:

Table I: Overview of the six repositories and their paraphrased counterparts (*Para. Name*) in RepoCraft. #Files denotes the total source files, LOC the effective lines of code, and Task Counts the evaluation tasks.

Real Repo	Para. Name	#Files	LOC	Task Counts
scikit-learn	MLKit-Py	185	65,972	236
pandas	TableKit	217	106,447	175
sympy	SymbolicMath	699	218,924	192
statsmodels	StatModeler	271	83,325	234
requests	HttpEasy	17	2,793	50
django	PyWebEngine	681	109,457	165

RQ1 (The Overall Effectiveness). How effective is Repo0 at zero-to-all code generation?

RQ2 (Ablation Study). How does each core design component of Repo0 contribute to zero-to-all code generation?

RQ3 (Structural Evolution Analysis). Do the proposed modularity metrics improve structural convergence compared with LLM-decided structural actions?

B. Datasets

We evaluate Repo0 on RepoCraft [11], a repository-generation benchmark consisting of six real-world Python repositories. Following RPG [11], repositories are exposed through paraphrased counterpart names to reduce potential pretraining leakage.

To better align the benchmark with the specific zero-to-all code generation task, namely, generating a repository from high-level requirements, we further ask two software engineers to revise the original task descriptions [19]. The revised descriptions remove repository-structure cues, implementation-specific architectural hints, and function-level interface details while preserving the original functional requirements.

The benchmark includes *scikit-learn*, *pandas*, *sympy*, *statsmodels*, *requests*, and *django*, exposed as *MLKit-Py*, *TableKit*, *SymbolicMath*, *StatModeler*, *HttpEasy*, and *PyWebEngine*, respectively. As summarized in Table I, these repositories span diverse software domains and scales, providing a challenging benchmark for evaluating repository generation across both compact and dependency-intensive systems.

C. Baseline Methods

We compare Repo0 with three representative baselines covering direct coding agents, staged repository generation, and graph-based repository planning. We additionally report the original ground-truth repository for each benchmark as a reference.

- **mini-SWE-agent** is a lightweight software engineering agent that solves coding tasks through iterative code editing and command execution [2], [27].
- **Paper2Code** is a staged multi-agent repository generation framework that decomposes generation into planning, analysis, and implementation phases [28].
- **RPG** is the state-of-the-art graph-based repository generation method, which constructs a static Repository Planning Graph to guide code generation [11].

Table II: RQ1 main results on three RepoCraft repositories. For each repository, we report Functionality Coverage (Cov.), Functionality Novelty (Nov.), and Pass./Vot., which combines Pass Rate and Voting Rate. **GPT-5 mini** marks the highest value among methods under GPT-5 mini, and **DeepSeek V3.2** marks the highest value among methods under DeepSeek V3.2.

Model	Method	requests			statsmodels			django		
		Cov. (%)	Nov. (%)	Pass./Vot. (%)	Cov. (%)	Nov. (%)	Pass./Vot. (%)	Cov. (%)	Nov. (%)	Pass./Vot. (%)
GPT-5 mini	mini-SWE-agent	68.18	2.63	4.11 / 27.40	18.18	9.09	0.00 / 31.86	47.92	6.96	37.04 / 44.44
	Paper2Code	95.50	7.20	24.66 / 24.66	44.32	24.13	4.42 / 30.09	66.67	15.30	30.04 / 78.60
	RPG	90.91	13.70	31.51 / 95.89	70.40	13.80	77.90 / 92.00	60.42	11.58	47.33 / 74.07
	Repo0	100.00	18.20	50.98 / 100.00	80.68	11.48	85.51 / 98.65	80.50	13.59	74.36 / 97.12
DeepSeek V3.2	mini-SWE-agent	86.36	15.34	21.92 / 47.95	59.09	23.39	2.65 / 53.98	33.33	9.38	10.70 / 47.33
	Paper2Code	90.91	11.80	4.11 / 56.16	14.77	5.00	49.56 / 61.95	62.50	43.46	7.82 / 53.50
	RPG	95.45	9.23	61.64 / 90.41	64.70	13.70	39.29 / 73.57	68.75	26.50	46.50 / 69.55
	Repo0	100.00	24.77	78.08 / 100.00	78.41	14.10	69.03 / 86.46	79.17	14.29	74.07 / 93.83
Human Developer	Gold Project	100.00	–	94.12 / 100.00	100	–	94.15 / 100.00	100.00	–	96.34 / 100.00

- **Gold Project** is the original repository corresponding to each benchmark task and serves as a reference for validating the evaluation pipeline.

D. Metrics

Following RPG [11], we adopt its evaluation pipeline and metric definitions.

Functionality Coverage measures the fraction of reference functional categories matched by at least one generated functionality description.

Functionality Novelty measures the fraction of generated functionalities that are not matched to any reference category under the same matching procedure.

Functionality Accuracy is evaluated using two task-level metrics: *Pass Rate*, the fraction of benchmark tasks whose adapted ground-truth tests pass on the generated repository, and *Voting Rate*, the fraction of tasks for which majority-vote semantic evaluation identifies a matched functional interface.

To mitigate evaluator bias, we adopt cross-model evaluation for both semantic voting and test-case rewriting. Specifically, DeepSeek V3.2 evaluates the repositories and rewritten test cases generated by GPT-5 mini, while GPT-5 mini evaluates those generated by DeepSeek V3.2.

E. Implementation Details

We use two backbone models in our experiments: the open-source DeepSeek V3.2 [29] and the close-source GPT-5 mini [30]. All generation runs use deterministic decoding with temperature set to 0. For Repo0, we repeat each experimental setting with three independent runs and report the averaged results.

For the structural evolution thresholds introduced in Section II, we set the split cohesion threshold to $\gamma_{\text{split}} = 2/3$ and the merge coupling threshold to $\theta_{\text{merge}} = 0.7$. These values are empirical settings selected on Commit0 Lite [10] using two held-out repositories, *wcwidth* (small-scale) and *sphinx* (large-scale), to cover repositories of different complexity. We compared the evolved component-level DAGs against the corresponding golden repository architectures and chose the setting that produced component boundaries closest to

the golden architecture by manually comparing the evolved architectures with the corresponding ground-truth repository architectures while avoiding excessive splitting or merging. The held-out repositories are not used in the RepoCraft evaluation.

To isolate the effect of repository-structure planning, each method uses its own method-specific procedure to construct the repository structure. Once the repository architecture is produced, all methods share the same downstream code-generation, validation, and repair scaffold under an identical test-driven development (TDD) execution protocol [25], [26].

All evaluation-pipeline hyperparameters follow the default RPG configuration [11].

Due to space constraints, the main experiment reports three representative RepoCraft repositories: *requests*, *statsmodels*, and *django*, which cover lightweight, medium-scale, and large-scale software projects respectively. All prompts and the results on the remaining three repositories are provided in the supplementary material².

IV. RESULTS

A. RQ1: The effectiveness of Repo0

Table II presents the main RQ1 results on three representative RepoCraft repositories. Across both backbone models and repositories of different scales, Repo0 consistently achieves the best overall repository-generation performance.

Under GPT-5 mini, Repo0 attains the highest *Functionality Coverage* on all three repositories, reaching 100.00% on *requests*, 80.68% on *statsmodels*, and 80.50% on *django*. The same pattern appears under DeepSeek V3.2, where Repo0 again achieves the best coverage across all evaluated repositories. This indicates that continuous structural evolution helps realize a broader portion of the required functionality.

Beyond covering more required functionality, Repo0 also translates these improvements into stronger implementation correctness. Repo0 also achieves the strongest implementation accuracy in most settings. Under GPT-5 mini, it obtains the highest *Pass Rate* on all three repositories, outperforming RPG

²<https://github.com/csloolow/Repo0/blob/main/supplementary.pdf>

by +19.47 points on *requests*, +7.61 points on *statsmodels*, and +27.03 points on *django*. For *Voting Rate*, Repo0 achieves the highest results in all six settings.

The baselines exhibit complementary weaknesses. mini-SWE-agent struggles to maintain repository-wide consistency on larger repositories, especially in *Pass Rate*. Paper2Code often produces relatively high *Functionality Novelty*, but these gains do not consistently translate into stronger correctness. RPG remains the strongest baseline overall, confirming the value of explicit repository-level planning, but it still degrades noticeably on more complex repositories.

Since all methods share the same downstream code-generation, validation, and repair pipeline, the primary difference lies in repository architecture. The consistent improvements therefore indicate that continuously refining repository architecture, rather than relying on a one-shot architectural plan, leads to both broader functionality realization and higher implementation correctness.

Finding for RQ1

Repo0 consistently achieves the strongest overall repository-generation performance across all baselines by a significant margin.

B. RQ2: Ablation Study

Table III reports the ablation results of Repo0. Removing any individual component degrades performance on at least one repository or evaluation metric, indicating that all four design choices contribute to the final repository-generation quality.

Among all ablations, removing Structural Evolution results in the largest overall performance degradation. Across all three repositories, it consistently reduces both functional coverage and implementation correctness, with particularly large drops on *requests* (*Coverage*: -5.68, *Pass Rate*: -8.47, *Voting Rate*: -17.86) and *django* (*Coverage*: -5.92, *Pass Rate*: -13.33, *Voting Rate*: -8.34). This suggests that repository generation benefits substantially from continuously evolving the architectural state, rather than treating the initially constructed architecture as fixed.

Removing Requirement Context primarily affects implementation correctness. During code generation, the full system conditions each component not only on its directly aligned sub-requirements, but also on the requirement nodes that are functionally coordinated with them through edges in the requirement-level DAG. This additional context helps the model preserve behavior consistency across related functionalities. Removing this contextual information consistently reduces *Pass Rate*, particularly on *django* (-10.00) and *requests* (-5.59), indicating that neighboring requirement-level context is important for producing implementations that correctly satisfy interacting functional behaviors.

The Component-Graph Ordering mainly influences implementation correctness rather than functional completeness. While *Functionality Coverage* remains unchanged on both re-

Table III: RQ2 ablation results on *requests*, *statsmodels*, and *django*.

Repository	Setting	Cov. (%)	Nov. (%)	Pass./Vot. (%)
<i>requests</i>	Repo0	100.00	18.20	50.98 / 100.00
	w/o Requirement Context	95.45 (-4.55)	9.05 (-9.15)	45.39 (-5.59) / 87.86 (-12.14)
	w/o Component-Graph Ordering	100.00 (0.00)	12.85 (-5.35)	50.98 (0.00) / 100.00 (0.00)
	w/o Dual-DAG	95.45 (-4.55)	9.14 (-9.06)	48.72 (-2.26) / 87.86 (-12.14)
	w/o Structural Evolution	94.32 (-5.68)	8.94 (-9.26)	42.51 (-8.47) / 82.14 (-17.86)
<i>statsmodels</i>	Repo0	80.68	11.48	85.51 / 98.65
	w/o Requirement Context	67.92 (-12.76)	11.69 (+0.21)	85.51 (0.00) / 98.65 (0.00)
	w/o Component-Graph Ordering	80.68 (0.00)	15.57 (+4.09)	55.51 (-30.00) / 88.65 (-10.00)
	w/o Dual-DAG	78.55 (-2.13)	14.66 (+3.18)	85.51 (0.00) / 95.32 (-3.33)
	w/o Structural Evolution	75.35 (-5.33)	13.50 (+2.02)	73.51 (-12.00) / 93.65 (-5.00)
<i>django</i>	Repo0	87.50	13.59	74.36 / 100.00
	w/o Requirement Context	78.29 (-9.21)	12.10 (-1.49)	64.36 (-10.00) / 100.00 (0.00)
	w/o Component-Graph Ordering	83.56 (-3.94)	11.58 (-2.01)	67.70 (-6.66) / 100.00 (0.00)
	w/o Dual-DAG	82.24 (-5.26)	12.40 (-1.19)	64.36 (-10.00) / 93.33 (-6.67)
	w/o Structural Evolution	81.58 (-5.92)	10.94 (-2.65)	61.03 (-13.33) / 91.66 (-8.34)

quests and *statsmodels*, the *Pass Rate* on *statsmodels* decreases by 30.00 points without dependency-aware generation order, demonstrating that generation order becomes increasingly important as repository complexity grows.

The Dual-DAG representation provides consistent improvements across repositories by separating requirement-level functional coordination from component-level implementation dependencies. Replacing the two-graph representation with a unified graph consistently reduces either functional coverage or implementation accuracy, suggesting that disentangling functional and implementation structures leads to more effective repository planning.

A secondary observation is that several ablated variants achieve slightly higher *Functionality Novelty* on *statsmodels*. Since this metric measures additional generated functionalities rather than successful realization of the benchmark functionality, the increased novelty together with reduced coverage suggests a tendency to generate extra functionalities at the expense of faithfully implementing the required repository behavior.

Finding for RQ2

All four design components positively contribute to the performance of Repo0, while Structural Evolution yields the largest overall improvements.

C. RQ3: Structural Convergence Analysis

a) **Metrics-Guided Convergence:** Figure 4 compares Repo0 with two alternatives on *statsmodels*: (i) *w/o evolution*, which directly uses the initial architectural state, and (ii) LLM-decided structural evolution with fixed budgets of 1, 3, and 5 rounds. Unlike Repo0, these budgeted variants allow the LLM to repeatedly perform structural actions without using the proposed cohesion and coupling metrics to determine which components should evolve or when structural convergence has been reached. In contrast, Repo0 guides *split* and *merge* using these metrics and terminates refinement once no metric-triggered structural update remains.

Overall, the full Repo0 setting achieves the best performance. Compared with *w/o evolution*, Repo0 improves all four metrics, increasing *Functionality Coverage* from 75.90% to 80.68%, *Functionality Novelty* from 11.05% to 11.48%,

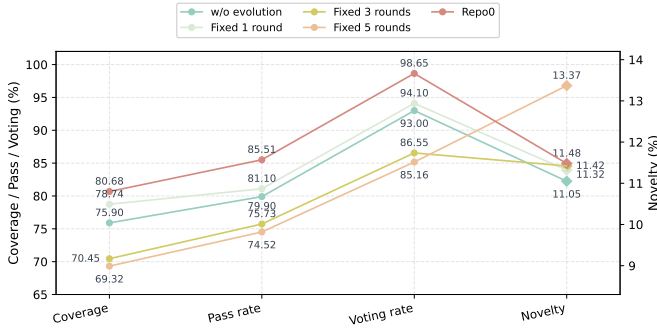


Fig. 4: RQ3 structural-convergence analysis on *statsmodels* with GPT-5 mini.

Pass Rate from 81.90% to 85.51%, and Voting Rate from 93.00% to 98.65%. More importantly, Repo0 also outperforms all budgeted LLM-decided variants, even though those variants are allowed additional structural action rounds. This suggests that the benefit does not simply come from applying more refinement steps; rather, the cohesion and coupling metrics help the component-level DAG reach structural convergence before code generation.

This behavior is consistent with the design of Repo0. The two modularity metrics, together with the split and merge decision rules defined in Section II, provide an explicit convergence criterion for the evolving component-level DAG. As refinement proceeds, components with diffuse requirement responsibility are split, components with overlapping responsibility are merged, and the loop stops once no further metric-triggered restructuring is needed. This enables subsequent code-generation decisions to operate on a progressively more stable architectural representation.

In contrast, LLM-decided structural evolution lacks this explicit convergence criterion. Repeated structural actions can continue beyond the point of structural convergence, leading to unnecessary fragmentation and compensatory restructuring. Although additional LLM-decided refinement may slightly increase novelty, it degrades coverage, pass rate, and voting rate after one round, indicating that unconstrained structural actions can move the repository away from coherent modular boundaries.

b) Action Distribution: Figure 5 summarizes the aggregated structural-action distribution during metrics-guided structural evolution across both backbone models. In both settings, *split* is the dominant action, followed by *save*, while *merge*, *revise*, and *add* occur less frequently. This indicates that structural evolution is primarily driven by localized updates to component boundaries rather than global changes to the component-level DAG. The relatively high proportion of *save* further suggests that many initial architectural states already contain stable components and require only limited modification.

We further observe systematic differences between backbone models, particularly in the frequency of *revise* and *add*. GPT-5 mini triggers these updates more often, and

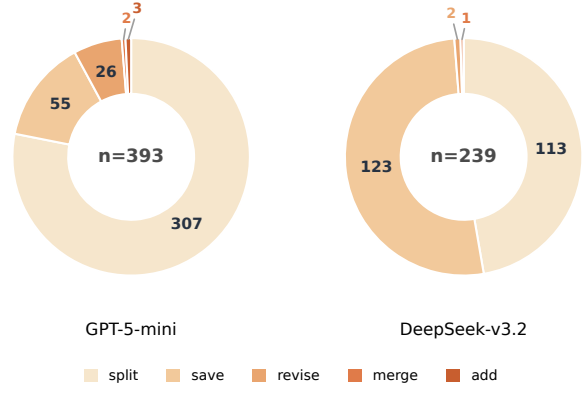


Fig. 5: Action distributions of different models across the six RepoCraft repositories during structural evolution.

manual inspection confirms that they correspond to valid updates to the architectural state: *revise* adjusts underspecified component responsibilities and interface assumptions, while *add* recovers missing requirements or components to improve requirement coverage. In contrast, DeepSeek V3.2 produces fewer such updates, which is consistent with a more complete initial architectural state before structural evolution.

To verify whether this difference is driven by the initial architectural state or the backbone model used during structural evolution, we fix the same unoptimized first-round architectural state generated by GPT-5 mini and apply structural evolution using both backbone models. Under this controlled setting, DeepSeek V3.2 produces nearly the same number of *revise* actions as GPT-5 mini does when evolving the GPT-5-mini-generated initial architectural state (see the supplementary material³). This suggests that the action distribution is largely determined by the initial architectural state generated by the LLM. *revise* actions are therefore especially important for LLMs that produce vague or underspecified initial component responsibilities and interface assumptions.

Finding for RQ3

Metrics-guided convergence improves repository quality beyond unconstrained LLM-decided structural actions, indicating that explicit modularity metrics help the architecture reach structural convergence.

D. Case Study

Figure 6 shows how Repo0 instantiates its Dual-DAG on StatModeler, the RepoCraft counterpart of *statsmodels*. Starting from a README that specifies capabilities such as regression modeling, generalized linear models, time-series analysis, statistical testing, data backends, result serialization, and model diagnostics, the system first extracts 39 high-level requirements as top-level decomposition

³<https://github.com/cslsolow/Repo0/blob/main/supplementary.pdf>

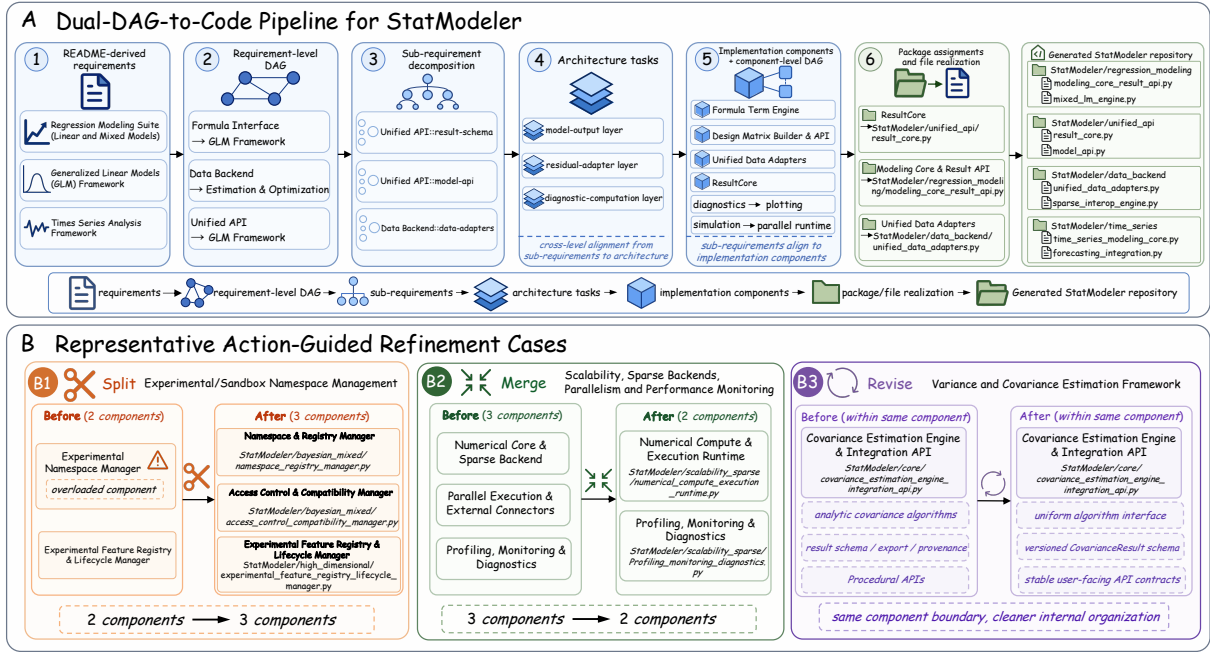


Fig. 6: Case study of Repo0 on StatModeler. The figure shows how requirements are decomposed, aligned with components, updated through structural actions, and materialized into files.

anchors. These anchors are decomposed and organized into a requirement-level DAG with 92 sub-requirements, such as Unified API::result-schema, Unified API::model-api, and Data Backend::data-adapters. On the implementation side, these sub-requirements are grounded into 86 implementation components, thereby forming the component-level DAG of the Dual-DAG. Throughout this process, the alignment relation explicitly connects sub-requirements to the components that realize them. Components such as ResultCore, Modeling Core & Result API, and Unified Data Adapters are ultimately materialized as concrete files including StatModeler/unified_api/result_core.py, StatModeler/regression_modeling/modeling_core_result_api.py, and StatModeler/data_backend/unified_data_adapters.py, illustrating how requirement-level structure is carried through to package and file realization.

The figure further highlights how the Dual-DAG remains revisable through structural actions during repository generation. Three representative structural-action cases are shown. A split action refines Experimental Namespace Manager into Namespace & Registry Manager and Access Control & Compatibility Manager, making the component boundary more explicit. A merge action consolidates Numerical Core & Sparse Backend and Parallel Execution & External Connectors into Numerical Compute & Execution Runtime, simplifying an over-segmented local structure in the component-level DAG. A revise action keeps the boundary of Covariance Estimation Engine & Integration API fixed while reorganizing its internals into a cleaner algorithm interface, a versioned CovarianceResult schema, and more stable user-facing API contracts. Together, these cases show that Repo0 does not treat

the Dual-DAG as a fixed blueprint: both component boundaries and the alignment relation can be updated through structural evolution before final code realization.

V. COST ANALYSIS

We analyze the monetary cost of repository generation on the three repositories reported in RQ1, namely *requests*, *statsmodels*, and *django*. Across both DeepSeek V3.2 and GPT-5 mini settings, we decompose costs into generation and evaluation phases to isolate where computational overhead arises.

Under DeepSeek V3.2, Repo0 incurs generation costs of \$11.95, \$28.19, and \$27.24 on *requests*, *statsmodels*, and *django*, respectively, while evaluation costs dominate for larger repositories, particularly *django* (\$72.82), yielding total costs of \$21.28, \$41.12, and \$100.06. In comparison, RPG exhibits substantially higher generation overhead on *requests* and *statsmodels* (+\$9.32 and +\$35.59, respectively), but lower cost on *django* (-\$8.83), indicating that Repo0 maintains lower end-to-end cost in most reported settings while improving repository-generation quality.

Under GPT-5 mini, Repo0 achieves consistently lower generation cost across all repositories, while evaluation costs remain comparable across methods. The complete cost details for all six repositories are provided in the supplementary material⁴. Further analysis shows that improving cohesion and reducing coupling lowers cost more directly by reducing the amount of redundant or conflicting code that needs to be generated and repaired. As a result, Repo0 requires fewer costly

⁴<https://github.com/cslosolow/Repo0/blob/main/supplementary.pdf>

TDD repair iterations caused by ambiguous or overlapping component responsibilities across both backbone models.

VI. THREATS TO VALIDITY

Internal Validity. A potential threat is that the quality of structural updates depends on the architectural reasoning capability of the backbone LLM. Although the modularity metrics select candidate `split` and `merge` actions, the LLM still instantiates these actions over the current architectural state by rewriting component boundaries, responsibility descriptions, alignment entries, and interface assumptions. This threat is partially mitigated by the iterative structural evolution loop: later evolution rounds can re-evaluate the component-level DAG and the alignment relation under the same metrics, while boundary-preserving `revise` actions can update component descriptions or interface assumptions without changing component boundaries. During TDD-based code generation, validation feedback can further expose inaccurate component descriptions or interface assumptions, triggering localized repair within the fixed repository architecture.

External Validity. Our experiments are conducted on six Python repositories from RepoCraft, covering diverse domains and repository scales, but they remain limited to a single programming language and benchmark. The effectiveness of structural evolution on other language ecosystems remains to be investigated. Nevertheless, the proposed architectural state, component-level structural actions, and modularity metrics operate at the level of repository architecture rather than language-specific syntax, suggesting that the framework is not inherently restricted to Python. Evaluating this generality on additional languages and benchmarks is left to future work.

VII. RELATED WORK

A. Benchmarks for Repository Generation

Repository generation [31]–[38] has recently emerged as a distinct benchmark setting beyond function-level code synthesis. DevBench evaluates broader software-engineering workflows, including implementation, testing, and debugging, but does not primarily target zero-to-repository generation [25], [39], [40]. Commit0 and NL2Repo-Bench move closer to repository-scale tasks, yet both still provide clear structural priors: Commit0 asks agents to complete missing code within a predefined repository architecture, file layout, and function interfaces [10], while NL2Repo-Bench includes the golden repository architecture and detailed module organization in its input specifications [14]. RepoCraft, introduced together with RPG, tightens the setting toward complete repository construction from high-level natural language requirements while reducing direct exposure to repository-specific implementation structure [11], [41]. RepoGenesis extends this line to multilingual microservice systems [12], and ProjDevBench studies end-to-end project development by modern coding agents [15]. These benchmarks progressively reduce structural priors available to repository-generation agents, highlighting the need for methods that can organize and refine repository architectures directly from high-level requirements. Our work

contributes to this line by using modularity principles to drive iterative optimization of repository architecture during generation.

B. Agents for Repository Generation

Repository-generation agents [42]–[62] have evolved from role-based workflows toward increasingly explicit structural reasoning. Early systems such as ChatDev, MetaGPT, and SoA coordinate specialized agents across staged development phases, but still rely primarily on natural language or predefined workflows as the carrier of intermediate planning [6], [7], [16]–[18], [63], [64]. Later work introduces more structured planning representations. CodeS decomposes repository generation into repository-, file-, and function-level sketches, enabling hierarchical repository construction through progressively refined planning artifacts [47]. Similarly, Paper2Code employs a multi-stage planning process that derives architectural designs and dependency structures before synthesizing runnable repositories from research papers [28]. EvoMAC further improves adaptability by dynamically evolving the multi-agent collaboration topology according to environmental feedback [65]. Despite these advances, repository architecture is still treated largely as a planning artifact that is generated once and subsequently executed.

RPG is the closest prior work to our setting. It introduces a Repository Planning Graph that explicitly represents capabilities, file structures, data flows, and functions, moving repository generation from purely natural-language planning toward graph-guided planning [11]. The key distinction is that RPG treats the planning graph as a blueprint for subsequent generation, whereas Repo0 treats repository architecture as a persistent and evolving state. Guided by modularity metrics, Repo0 continuously updates component boundaries and alignments toward higher cohesion and lower coupling.

VIII. CONCLUSION

This paper presents Repo0, a continuous structural evolution framework for repository generation from high-level natural-language requirements. Unlike prior methods that rely on a fixed repository plan, Repo0 maintains an explicit architectural state and iteratively refines repository structure to improve modularity throughout generation. Experiments on RepoCraft demonstrate that Repo0 consistently outperforms representative baselines across both open-source and closed-source backbone models. Ablation studies further show that each major component contributes to overall performance, while metrics-guided structural convergence is critical for effective repository evolution. Overall, our findings suggest that successful repository generation requires not only code synthesis, but also explicit architectural reasoning and continuous structural refinement. We hope this work encourages future research on architecture-aware agents for long-horizon software engineering tasks.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig *et al.*, “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.
- [2] J. Yang, C. E. Jimenez, A. Wettig *et al.*, “Swe-agent: Agent-computer interfaces enable automated software engineering,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>
- [3] R. Bairei, A. Sonwane, A. Kanade *et al.*, “Codeplan: Repository-level coding using llms and planning,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.12499>
- [4] X. Wang, B. Li, Y. Song *et al.*, “Openhands: An open platform for ai software developers as generalist agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2407.16741>
- [5] W. Peng, Y. Shi, Y. Wang, X. Zhang, B. Shen, and X. Gu, “Swe-qa: Can language models answer repository-level code questions?” *arXiv preprint arXiv:2509.14635*, 2025.
- [6] S. Chen, S. Lin, Y. Shi *et al.*, “Swe-exp: Experience-driven software issue resolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2507.23361>
- [7] D. Ma, S. Chen, Y. Yang, Y. Shi, Y. Yan, and X. Gu, “Llm agents can see code repositories,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.14061>
- [8] J. Li, H. Zhu, H. Liu *et al.*, “Aligning llms to fully utilize the cross-file context in repository-level code completion,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 1477–1489.
- [9] Y. Shi, J. Xu, K. Fu *et al.*, “Swe-bench promax: Benchmarking agents on large-scale multilingual code refactoring,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.09802>
- [10] W. Zhao, N. Jiang, C. Lee *et al.*, “Commit0: Library generation from scratch,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2412.01769>
- [11] J. Luo, X. Zhang, S. Liu *et al.*, “Rpg: A repository planning graph for unified and scalable codebase generation,” 2026. [Online]. Available: <https://arxiv.org/abs/2509.16198>
- [12] Z. Peng, X. Yin, P. Zhao *et al.*, “Repogenesis: Benchmarking end-to-end microservice generation from readme to repository,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.13943>
- [13] E. Yourdon and L. L. Constantine, *Structured design: fundamentals of a discipline of computer program and systems design*. Prentice-Hall, Inc., 1979.
- [14] J. Ding, S. Long, C. Pu *et al.*, “Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2512.12730>
- [15] P. Lu, S. Zhang, Y. Hou *et al.*, “Projdevbench: Benchmarking ai coding agents on end-to-end project development,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.01655>
- [16] C. Qian, W. Liu, H. Liu *et al.*, “Chatdev: Communicative agents for software development,” 2024. [Online]. Available: <https://arxiv.org/abs/2307.07924>
- [17] S. Hong, M. Zhuge, J. Chen *et al.*, “Metagtpt: Meta programming for a multi-agent collaborative framework,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.00352>
- [18] Y. Ishibashi and Y. Nishimura, “Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization,” *arXiv preprint arXiv:2404.02183*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.02183>
- [19] J. Luo, C. Yin, X. Zhang *et al.*, “Closing the loop: Universal repository representation with rpg-encoder,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.02084>
- [20] F. Teng, Q. Shi, Z. Yu *et al.*, “Atom of thoughts for markov llm test-time scaling,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 74 010–74 040, 2026.
- [21] C.-E. Bichot and P. Siarry, *Graph partitioning*. John Wiley & Sons, 2013.
- [22] D. Wagner and F. Wagner, “Between min cut and graph bisection,” in *International Symposium on Mathematical Foundations of Computer Science*. Springer, 1993, pp. 744–750.
- [23] W. P. Stevens, G. J. Myers, and L. L. Constantine, “Structured design,” *IBM systems journal*, vol. 13, no. 2, pp. 115–139, 1974.
- [24] R. Real and J. M. Vargas, “The probabilistic basis of jaccard’s index of similarity,” *Systematic biology*, vol. 45, no. 3, pp. 380–385, 1996.
- [25] P. Chang, Y. Fang, S. Chen, Y. Shi, B. Shen, and X. Gu, “Test vs mutant: Adversarial llm agents for robust unit test generation,” *arXiv preprint arXiv:2602.08146*, 2026.
- [26] K. Beck, *Test-driven development: by example*. Addison-Wesley Professional, 2003.
- [27] Mini SWE Agent, “mini-swe-agent,” <https://mini-agent.ai/>, 2026, accessed: 2026-05-07.
- [28] M. Seo, J. Baek, S. Lee, and S. J. Hwang, “Paper2code: Automating code generation from scientific papers in machine learning,” *arXiv preprint arXiv:2504.17192*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.17192>
- [29] DeepSeek-AI, A. Liu, A. Mei *et al.*, “Deepseek-v3.2: Pushing the frontier of open large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.02556>
- [30] A. Singh, A. Fry, A. Perelman *et al.*, “Openai gpt-5 system card,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.03267>
- [31] R. Hu, C. Peng, J. Xu, and C. Gao, “Repo2run: Automated building executable environment for code repository at scale,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 32 679–32 718, 2026.
- [32] G. Orlanski, D. Roy, A. Yun *et al.*, “Slopcodebench: Benchmarking how coding agents degrade over long-horizon iterative tasks,” *arXiv preprint arXiv:2603.24755*, 2026.
- [33] S. Wang, Z. Wang, D. Ma *et al.*, “Codeflowbench: A multi-turn, iterative benchmark for complex code generation,” *arXiv preprint arXiv:2504.21751*, 2025.
- [34] C. Dilgren, P. Chiniya, L. Griffith, Y. Ding, and Y. Chen, “Secrepobench: Benchmarking llms for secure code generation in real-world repositories,” *arXiv e-prints*, pp. arXiv–2504, 2025.
- [35] S. Abedu, L. Menneron, S. Khatoonabadi, and E. Shihab, “Reepochat: An llm-powered chatbot for github repository question-answering,” in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 2025, pp. 255–259.
- [36] Z. Jiang, L. Deng, J. Cao, M. Pradel, and Z. Liu, “Doc2feat-bench: Evaluating documentation-driven feature addition,” 2026. [Online]. Available: <https://arxiv.org/abs/2507.18130>
- [37] K. Wang, P. Lan, J. Liu *et al.*, “Code refinement with repository context: How far are we?” *ACM Trans. Softw. Eng. Methodol.*, Aug. 2026, just Accepted. [Online]. Available: <https://doi.org/10.1145/3820059>
- [38] Z. Sun, X. Du, Z. Yang, L. Li, and D. Lo, “Ai coders are among us: Rethinking programming language grammar towards efficient code generation,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 1124–1136. [Online]. Available: <https://doi.org/10.1145/3650212.3680347>
- [39] B. Li, W. Wu, Z. Tang *et al.*, “Prompting large language models to tackle the full software development lifecycle: A case study,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.08604>
- [40] Y. Wang, Z. Wang, Y. Shi *et al.*, “Context compression for llm agents: A survey of methods, failure modes, and evaluation,” *Preprints*, May 2026. [Online]. Available: <https://doi.org/10.20944/preprints202605.2065.v1>
- [41] Y. Zhang, C. Wan, and B. Jin, “An empirical study on recovering requirement-to-code links,” in *2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. IEEE, 2016, pp. 121–126.
- [42] X. Gu, M. Chen, Y. Lin *et al.*, “On the effectiveness of large language models in domain-specific code generation,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 3, pp. 1–22, 2025.
- [43] L. Zhang, S. He, C. Zhang *et al.*, “Swe-bench goes live!” 2025. [Online]. Available: <https://arxiv.org/abs/2505.23419>
- [44] L. Fan, J. Liu, Z. Liu, D. Lo, X. Xia, and S. Li, “Exploring the capabilities of llms for code-change-related tasks,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 6, pp. 1–36, 2025.
- [45] M. Wen, J. Chen, R. Wu, D. Hao, and S.-C. Cheung, “Context-aware patch generation for better automated program repair,” in *Proceedings of the 40th international conference on software engineering*, 2018, pp. 1–11.
- [46] F. Zhang, B. Chen, Y. Zhang *et al.*, “Repocoder: Repository-level code completion through iterative retrieval and generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484.

- [47] D. Zan, A. Yu, W. Liu *et al.*, “Codes: Natural language to code repository via multi-layer sketch,” *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 7, Jun. 2026. [Online]. Available: <https://doi.org/10.1145/3768577>
- [48] C. Hu, W. Zeng, Y. Shi, B. Shen, and X. Gu, “In line with context: Repository-level code generation via context inlining,” *arXiv preprint arXiv:2601.00376*, 2026.
- [49] W. Zeng, Y. Wang, C. Hu *et al.*, “Pruning the unsurprising: Efficient code reasoning via first-token surprisal,” *arXiv preprint arXiv:2508.05988*, 2025.
- [50] W. Zeng, Y. Shi, X. Gu *et al.*, “Dockerless: Environment-free program verifier for coding agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.28436>
- [51] W. Zeng, X. Zhang, Y. Shi *et al.*, “Glimprouter: Efficient collaborative inference by glimpsing one token of thoughts,” *arXiv preprint arXiv:2601.05110*, 2026.
- [52] S. Gao, W. Zeng, Z. Yu *et al.*, “Swe-mem: Learning adaptive memory management for long-horizon coding agents,” *arXiv preprint arXiv:2606.28434*, 2026.
- [53] J. Huang, S. Yun, S. Chen, X. Gu, and B. Shen, “Planning over actions: Agentic reasoning for semi-structured table question answering,” *Information Processing & Management*, vol. 64, no. 1, p. 105092, 2027.
- [54] H. Lin, S. Chen, X. Gu *et al.*, “Know before fix: Qa-driven repository knowledge acquisition for software issue resolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.11111>
- [55] S. Chen, H. Li, X. Gu, Y. Shi, and H. Guan, “Skillforge: Self-distilling agents for project-specific issue resolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.18933>
- [56] Y. Shi, Y. Qian, H. Zhang, B. Shen, and X. Gu, “Longcodezip: Compress long context for code language models,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Press, 2025, p. 141–153. [Online]. Available: <https://doi.org/10.1109/ASE63991.2025.00020>
- [57] Y. Shi, S. Wang, C. Wan, M. Wang, and X. Gu, “From code to correctness: Closing the last mile of code generation with hierarchical debugging,” 2025. [Online]. Available: <https://arxiv.org/abs/2410.01215>
- [58] Z. Liu, Z. Jiang, Z. Ye, H. Wang, J. Liu, and X. Ren, “Effective and efficient context retrieval via partial dependency graph for repository-level code generation,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.01927>
- [59] F. Rabbi, Z. Ding, and J. Yang, “A multi-language perspective on the robustness of llm code generation,” 2026. [Online]. Available: <https://arxiv.org/abs/2504.19108>
- [60] Z. Lin, M. Zhou, Z. Sun *et al.*, “Reporecue: An empirical study of llm agents on whole-repository compatibility rescue,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.01213>
- [61] Y. Li, S. Liu, K. Chen, T. Zhang, and Y. Liu, “Impact-driven context filtering for cross-file code completion,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.05970>
- [62] Y. Qin, H. Wang, C. Xu, X. Ma, and J. Lu, “Syneva: Evaluating ml programs by mirror program synthesis,” in *2018 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2018, pp. 171–182.
- [63] Y. Lin, Y. Ma, R. Cao *et al.*, “Llms as continuous learners: Improving the reproduction of defective code in software issues,” *arXiv preprint arXiv:2411.13941*, 2024.
- [64] R. Pan, J. Wang, Q. Zhang *et al.*, “Persistent cross-attempt state optimization for repository-level code generation,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.03632>
- [65] Y. Hu, Y. Cai, Y. Du *et al.*, “Self-evolving multi-agent collaboration networks for software development,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 23 007–23 039.